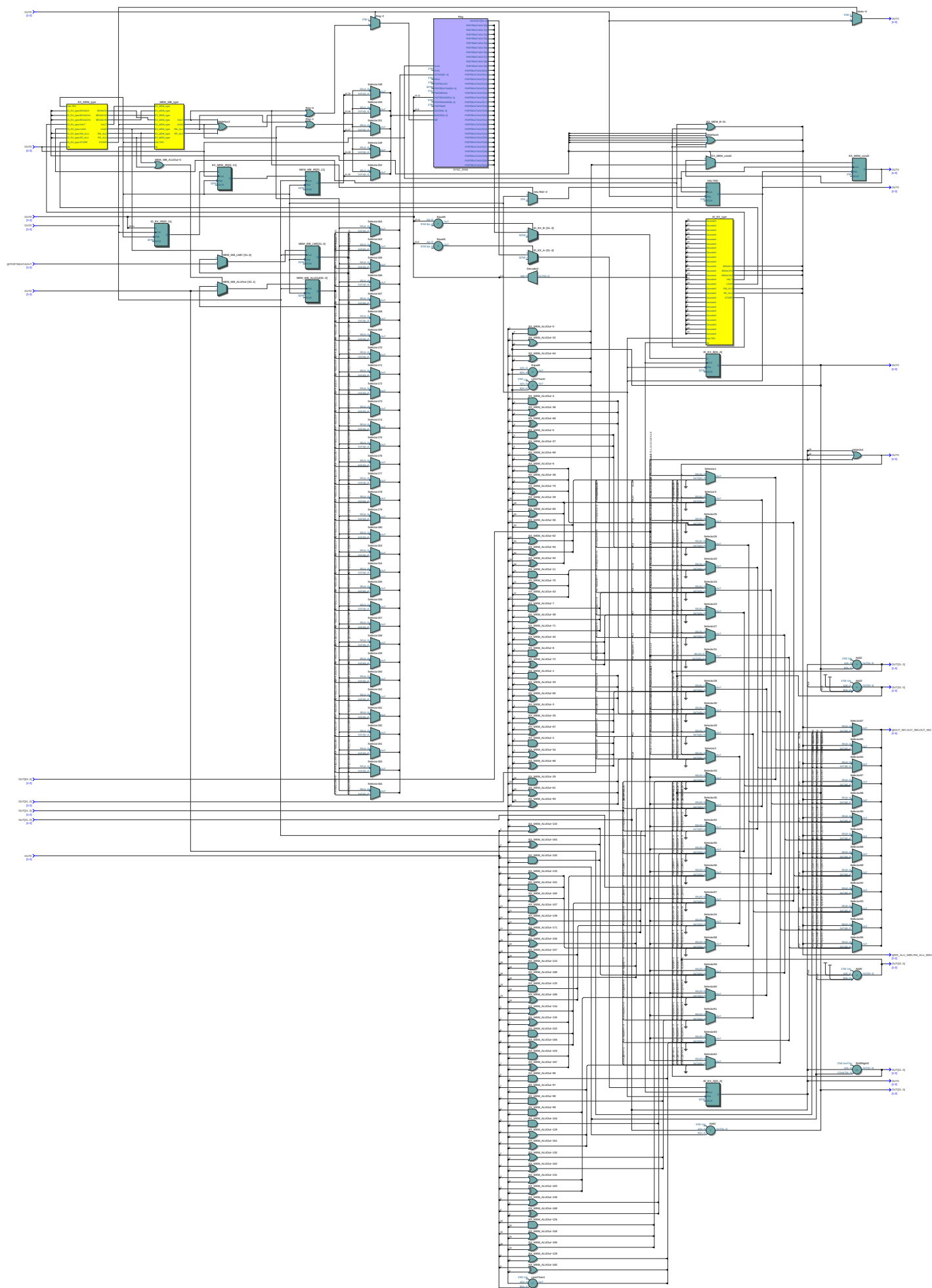
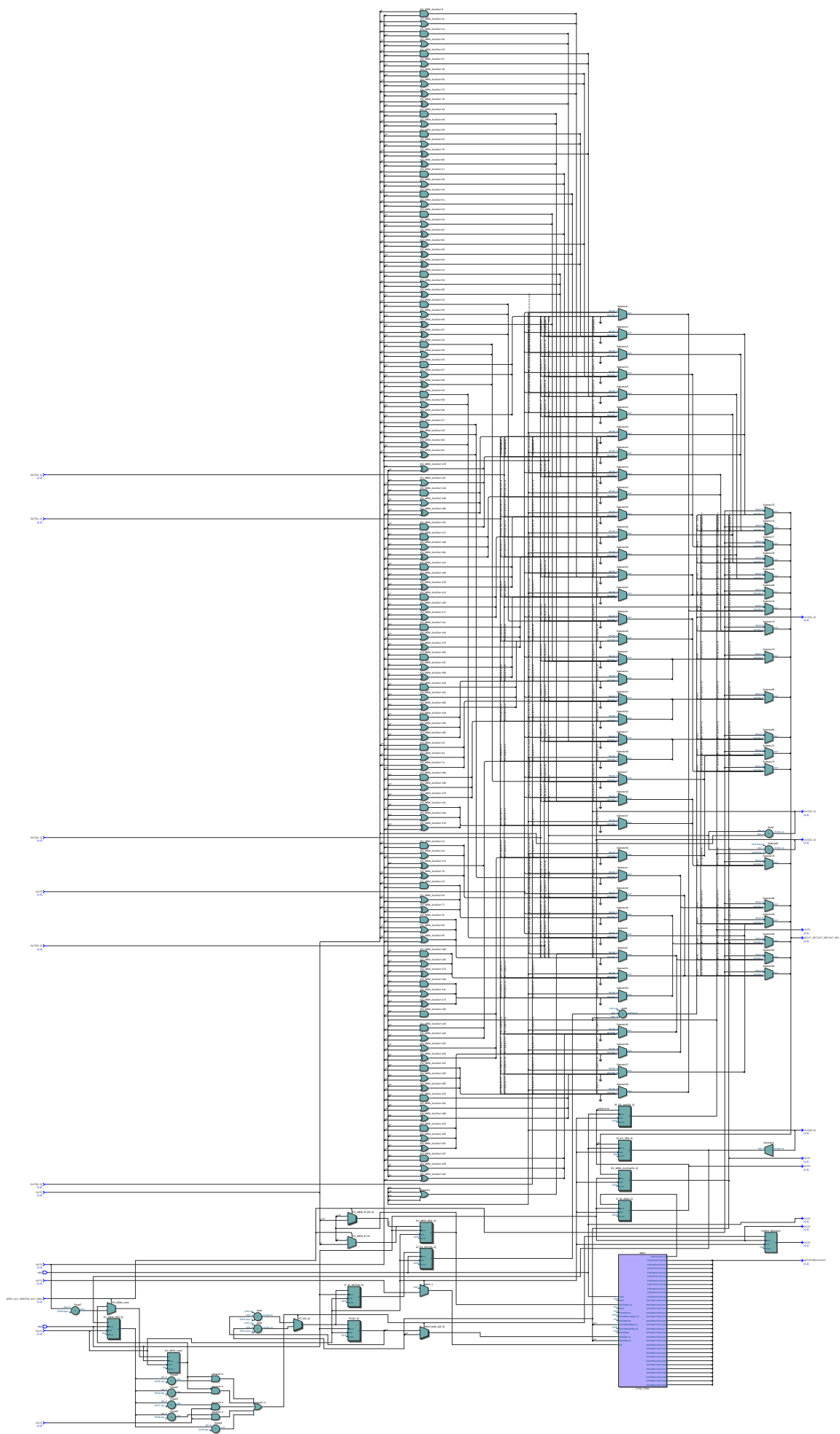
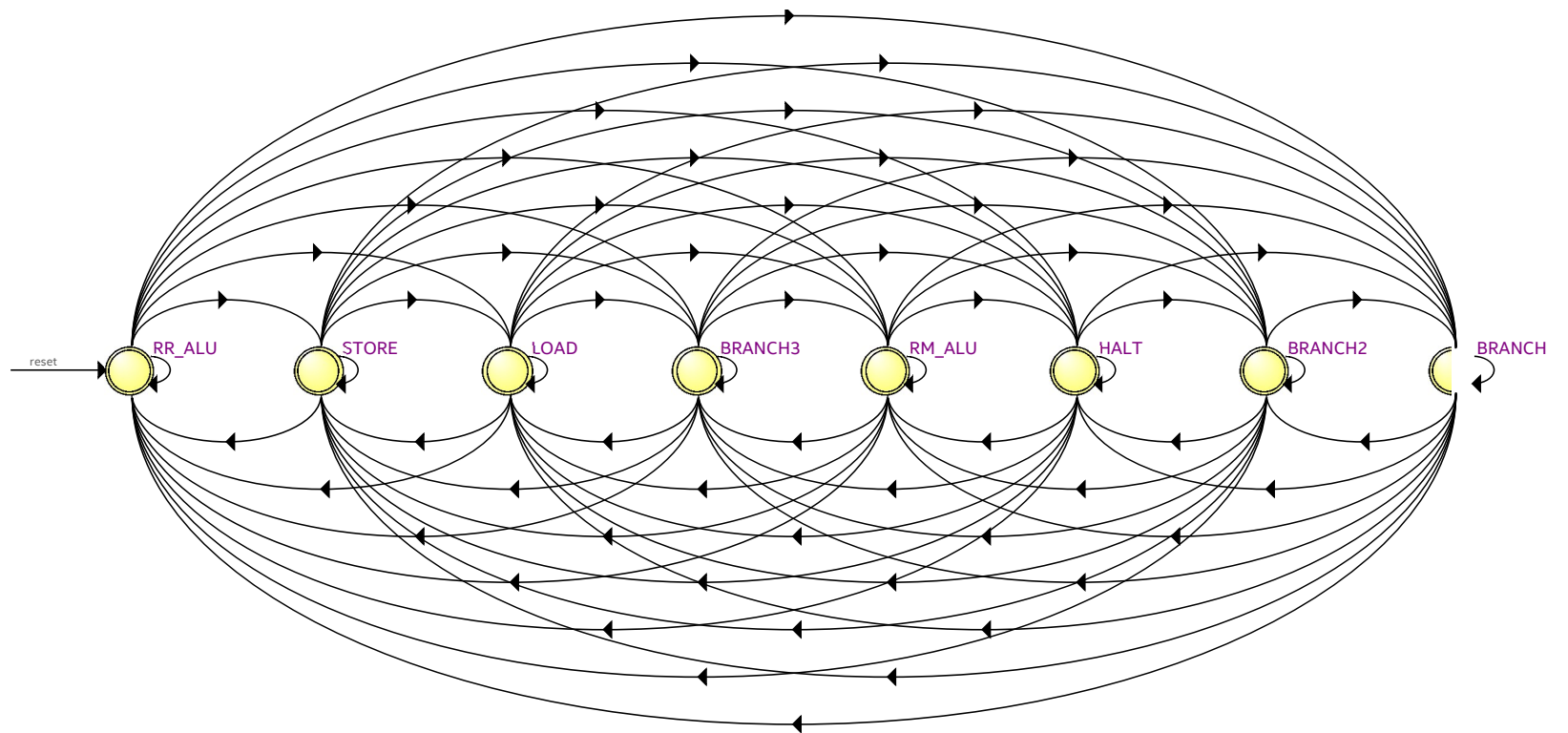








INSTRUCTIONS SET

S.NO	INSTRUCTIONS	OPCODE	HEXCODE
1.	Addition of two registers	ADD	6'b000000
2.	Addition with immediate data	ADDI	6'b000001
3.	Subtraction of two registers	SUB	6'b000010
4.	Subtractions with immediate data	SUBI	6'b000011
5.	Bitwise logical AND operation	AND	6'b000100
6.	Bitwise logical OR operation	OR	6'b000101
7.	Bitwise logical XOR operation	XOR	6'b000110
8.	Bitwise logical AND operation with immediate data	ANDI	6'b000111
9.	Bitwise logical OR operation with immediate data	ORI	6'b001000
10.	Set if less than (on comparing with registers)	SLT	6'b001001
11.	Set if less than (on comparing with immediate data)	SLTI	6'b001010
12.	Multiplications two registers	MUL	6'b001011
13.	Shift left by one bit	SLL	6'b001100
14.	Shift left by immediate data	SLLI	6'b001101
15.	Shift right by one bit	SRL	6'b001110
16.	Shift right by immediate data	SRLI	6'b001111
17.	Load the immediate data into register	LW	6'b010000
18.	Store the register into immediate data	SW	6'b010001
19.	Set if equal (on comparing with registers)	SEQ	6'b010010
20.	Set if not equal (on comparing with registers)	SNEQ	6'b010011
21.	Taken Branch if not equal to zero(register)	BNEQZ	6'b010100
22.	Taken Branch if equal to zero(register)	BEQZ	6'b010101
23.	Taken Branch if two registers are equal	BEQ	6'b010110
24.	Taken Branch if two registers are not equal	BNEQ	6'b010111
25.	Bitwise logical XOR operation with immediate data	XORI	6'b011000
26.	Jump to immediate address	JMP	6'b011001
27.	Halt :end the program	HLT	6'b111111

TESTBENCH FOR ALU OPERATIONS ON 32bits PROCESSOR

```
module mp_risc_tb_1;

    reg clk1, clk2;

    integer k;

    mp_risc mips (clk1, clk2);

    // Generating two-phase clock

    initial
    begin
        clk1 = 0; clk2 = 0;

        repeat (20)
        begin
            #5 clk1 = 1; #5 clk1 = 0; // Positive-edge clock1, negative-edge clock1
            #5 clk2 = 1; #5 clk2 = 0; // Positive-edge clock2, negative-edge clock2
        end
    end

    // Initializing the MIPS processor memory and registers

    initial
    begin
        for (k=0; k<31; k=k+1)
            mips.Reg[k] = k; // Assigning values to general-purpose registers

        mips.Mem[200] = 32'h00000005; // Memory location 200 initialized with 5
        mips.Mem[0] = 32'h00c80810; // Memory location 0 initialized with instruction (LW R1,00C8)
        mips.Mem[1] = 32'h000739c5; // Memory location 1 initialized with instruction (OR R7,R7,R7)---DUMMY VARIABLE
        mips.Mem[2] = 32'h000739c5; // Memory location 2 initialized with instruction (OR R7,R7,R7)---DUMMY VARIABLE
        mips.Mem[3] = 32'h000a1041; // Memory location 3 initialized with instruction (ADDI R2,R1,000A)
        mips.Mem[4] = 32'h00041841; // Memory location 4 initialized with instruction (ADDI R3,R1,0004)
        mips.Mem[5] = 32'h000739c5; // Memory location 5 initialized with instruction (OR R7,R7,R7)---DUMMY VARIABLE
        mips.Mem[6] = 32'h000739c5; // Memory location 6 initialized with instruction (OR R7,R7,R7)---DUMMY VARIABLE
        mips.Mem[7] = 32'h00011889; // Memory location 7 initialized with instruction (SLT R1,R2,R3)
        mips.Mem[8] = 32'h0004188b; // Memory location 8 initialized with instruction (MUL R4,R2,R3)
        mips.Mem[9] = 32'h00051884; // Memory location 9 initialized with instruction (AND R5,R2,R3)
        mips.Mem[10] = 32'h0001188c; // Memory location 10 initialized with instruction (SLL R1,R2)
        mips.Mem[11] = 32'h00041893; // Memory location 11 initialized with instruction (SNEQ R4,R2,R3)
```

```

mips.Mem[12] = 32'h00051886; // Memory location 12 initialized with instruction (XOR R5,R2,R3)
mips.Mem[13] = 32'h00110888; // Memory location 13 initialized with instruction (ORI R1,R2,0001)
mips.Mem[14] = 32'h00c91011; // Memory location 14 initialized with instruction (SW R2,00C9)
mips.Mem[15] = 32'h00ca1811; // Memory location 15 initialized with instruction (SW R3,00CA)
mips.Mem[16] = 32'h0000003f; // Memory location 16 initialized with instruction (HALT)


mips.HALTED = 0; // Initialize HALTED signal to 0 (not halted)
mips.PC = 0; // Initialize program counter (PC) to 0
mips.TAKEN_BRANCH = 0; // Initialize TAKEN_BRANCH signal to 0

#280
for (k=0; k<6; k=k+1)
    $display ("R%1d - %2d", k, mips.Reg[k]); // Display the initial values of some registers
end

initial
begin
    $dumpfile ("mips.vcd"); // Define VCD dumpfile
    $dumpvars (0, mp_risc_tb_1); // Dump variables for VCD
    #300 $finish; // Finish the simulation after some delay
end
endmodule

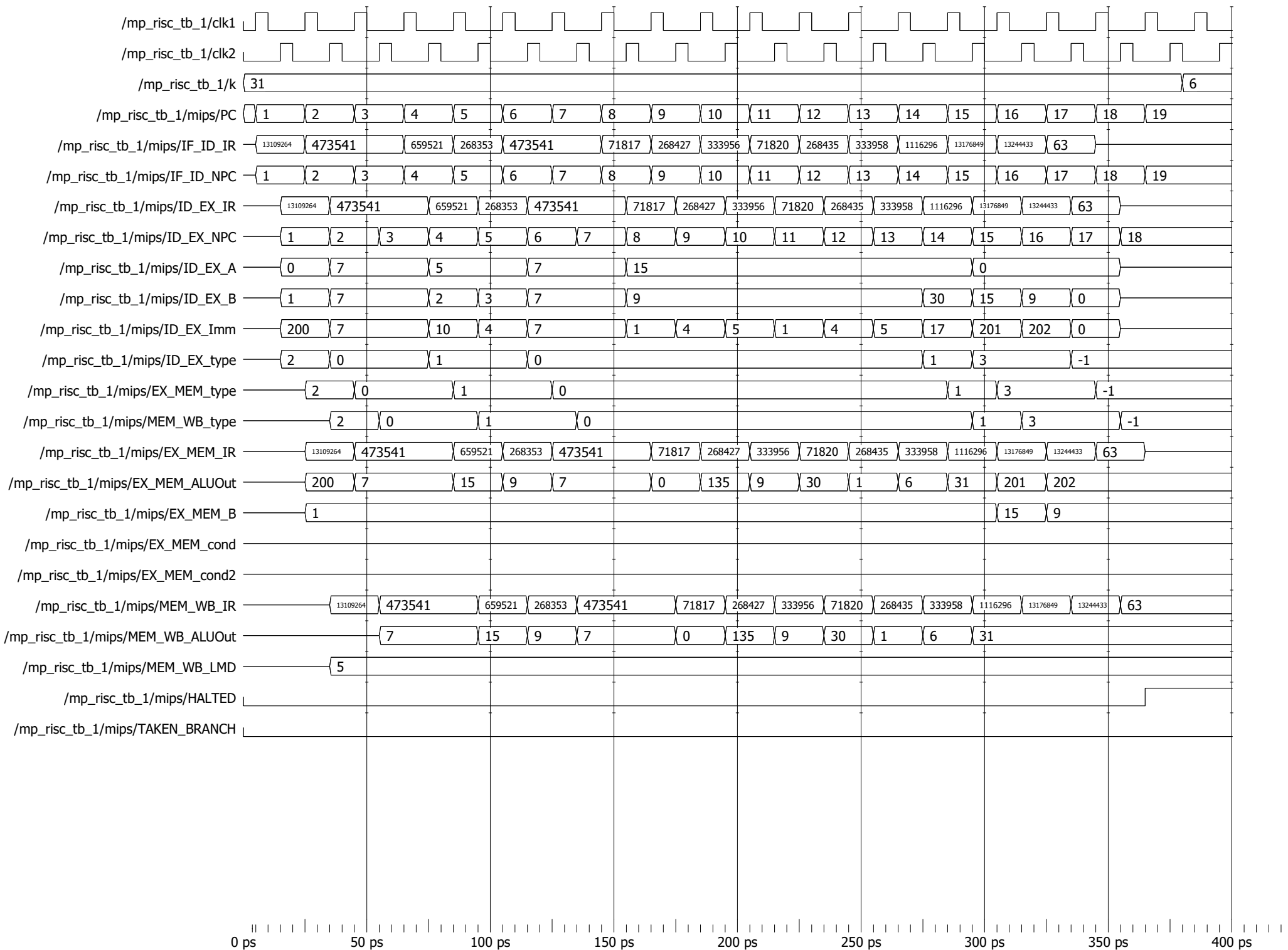
```

ASSEMBLY LANGUAGE PROGRAM:

```

LW  R1,00C8;
ADDI R2,R1,000A;
ADDI R3,R1,0004;
SLT  R1,R2,R3;
MUL  R4,R2,R3;
AND  R5,R2,R3;
SLL  R1,R2;
SNEQ R4,R2,R3;
XOR  R5,R2,R3;
ORI  R1,R2,0001;
SW   R2,00C9;
SW   R3,00CA;
HLT

```



TESTBENCH FOR COMPUTE THE FACTORIAL OF N

```
module mp_risc_tb_2;

    reg clk1, clk2;

    integer k;

    mp_risc mips (clk1, clk2);

    // Generating two-phase clock

    initial
    begin
        clk1 = 0; clk2 = 0;

        repeat (50)
        begin
            #5 clk1 = 1; #5 clk1 = 0; // Positive-edge clock1, negative-edge clock1
            #5 clk2 = 1; #5 clk2 = 0; // Positive-edge clock2, negative-edge clock2
        end
    end

    // Initializing the MIPS processor memory and registers

    initial
    begin
        for (k=0; k<31; k=k+1)
            mips.Reg[k] = k; // Assigning values to general-purpose registers

        mips.Mem[200] = 7;      // Memory location 200 initialized with 7 (factorial input)
        mips.Mem[0] = 32'h00c81810; // Memory location 0 initialized with instruction (LW R3,00C8)
        mips.Mem[1] = 32'h00011001; // Memory location 1 initialized with instruction (ADDI R2,R0,1)
        mips.Mem[2] = 32'h000739c5; // Memory location 2 initialized with instruction (OR R7,R7,R7)—DUMMY VARIABLE
        mips.Mem[3] = 32'h0002188b; // Memory location 3 initialized with instruction (LOOP: MUL R2,R2,R3)
        mips.Mem[4] = 32'h000118c3; // Memory location 4 initialized with instruction (SUBI R3,R3,1)
        mips.Mem[5] = 32'h000739c5; // Memory location 5 initialized with instruction (OR R7,R7,R7) —DUMMY VARIABLE
        mips.Mem[6] = 32'hffc00d4; // Memory location 6 initialized with instruction (BNEQZ R3,LOOP, -4 OFFSET)
        mips.Mem[7] = 32'h00c91011; // Memory location 7 initialized with instruction (SW R2,00C9)
        mips.Mem[8] = 32'h0000003f; // Memory location 8 initialized with instruction (HALT)

        mips.PC = 0;      // Initialize program counter (PC) to 0
        mips.HALTED = 0;  // Initialize HALTED signal to 0 (not halted)
```



```

mips.TAKEN_BRANCH = 0; // Initialize TAKEN_BRANCH signal to 0

#2000 $display ("Mem[200] = %2d, Mem[201] = %6d", mips.Mem[200], mips.Mem[201]); // Display initial memory
values
end

// VCD dump and monitor
initial
begin
    $dumpfile ("mips.vcd"); // Define VCD dumpfile
    $dumpvars (0, mp_risc_tb_2); // Dump variables for VCD
    $monitor ("R2: %4d", mips.Reg[2]); // Monitor register R2
    #3000 $finish; // Finish the simulation after some delay
end
endmodule

```

ASSEMBLY LANGUAGE PROGRAM:

```

        LW      R3,00C8;
        ADDI    R2,R0,1;
LOOP:   MUL     R2,R2,R3;
        SUBI    R3,R3,1;
        BNEQZ   R3,LOOP;
        SW      R2,00C9;
        HLT

```

